

Preliminary Analysis of Developer Activity and Language Ecosystems in a Large GitHub Follow Network

Flyxion

March 16, 2026

Abstract

This report presents preliminary findings from a large-scale observational study of developer activity within a GitHub follow network. Using the GitHub GraphQL API, we collect activity metadata for accounts followed by a single user whose follow graph exceeds one million accounts. Each account is classified as active or inactive based on the recency of pushes to public repositories, and the dominant programming language of the most recently updated repository is recorded.

Analysis of the first $\sim 150,000$ accounts reveals systematic differences in activity levels across programming language ecosystems. The overall active fraction is approximately 0.589. Newer ecosystems such as Astro, Rust, and TypeScript exhibit activity ratios above 0.83, while older ecosystems such as Ruby and Java fall near 0.44–0.51. Unexpectedly, the developer survival curve is not monotonically decreasing: active fractions decline from account ages zero through eleven years, reaching a minimum near 0.52, and then *rise* steadily for accounts aged twelve to seventeen years, recovering to fractions above 0.66. This U-shaped pattern suggests that the oldest GitHub accounts in the dataset are disproportionately held by developers who have remained continuously active since the platform’s earliest years. These results provide an empirical snapshot of the structure and generational stratification of open source developer communities.

1 Introduction

GitHub has become one of the central platforms for collaborative software development, and its public data provides a unique opportunity to study the dynamics of programming language ecosystems and developer activity over time.

Most existing studies rely on repository-level datasets such as GitHub Archive or BigQuery mirrors of repository metadata. While these datasets capture repository events in detail, they provide only indirect insight into the life cycles of individual developers.

The present study instead analyzes a *developer-centered dataset*. Starting from a single GitHub account that follows more than one million other accounts, we crawl the follow network using the GitHub GraphQL API and collect basic metadata describing each followed account. The goal is to estimate:

- the fraction of developers who remain active over time,
- differences in activity rates across programming language ecosystems,
- and the approximate lifetime of developer participation on GitHub.

The results reported here are preliminary and based on the first $\sim 150,000$ accounts processed during an ongoing crawl.

2 Dataset Construction

The dataset is collected using the GitHub GraphQL API. For each followed account we retrieve:

- account creation timestamp,
- number of followers,
- number of public repositories,
- the most recently pushed public repository,
- and the primary language of that repository.

Accounts are classified as **active** if the most recent repository push occurred within the previous year; otherwise the account is classified as **inactive**. Accounts for which no language-tagged repository is available appear in the dataset under the label NONE.

For the $N = 154,283$ accounts processed at the time of this report, the dataset contains:

Active accounts	90,808
Inactive accounts	63,475
Active fraction	≈ 0.589

Because the sample is drawn from a follow network rather than uniformly across GitHub, it is biased toward more visible and historically active developers. This selection effect is discussed further in Section 4.

3 Language Activity Ratios

For each programming language we compute an *activity ratio*

$$R = \frac{\text{active}}{\text{active} + \text{inactive}}.$$

This metric estimates the probability that a developer whose most recently active repository is tagged with a given language is still committing code. Table 1 reports results for all languages with at least 336 accounts in the dataset, ordered by activity ratio.

Language	Active	Inactive	Total	R
Astro	306	39	345	0.89
MDX	303	48	351	0.86
Rust	2160	400	2560	0.84
TypeScript	11988	2413	14401	0.83
Lua	692	169	861	0.80
Dart	588	221	809	0.73
Go	2460	991	3451	0.71
Python	16890	6973	23863	0.71
Shell	2242	1011	3253	0.69
Kotlin	696	326	1022	0.68
HCL	239	119	358	0.67
HTML	6617	3320	9937	0.67
SCSS	401	200	601	0.67
PowerShell	237	126	363	0.65
Dockerfile	217	119	336	0.65
TeX	284	166	450	0.63
Swift	565	331	896	0.63
R	677	421	1098	0.62
Vue	514	329	843	0.61
JavaScript	8531	5357	13888	0.61
Jupyter Notebook	3749	2486	6235	0.60
C++	2757	1822	4579	0.60
C#	1444	1075	2519	0.57
C	1619	1222	2841	0.57
CSS	1227	1008	2235	0.55
PHP	1154	987	2141	0.54
Java	2649	2502	5151	0.51
MATLAB	189	187	376	0.50
Ruby	674	848	1522	0.44
NONE	14682	26193	40875	0.36

Table 1: Activity ratios for languages with at least 336 accounts ($N = 154,283$). Ordered by activity ratio descending; NONE placed last.

Several patterns are immediately visible.

Newer language ecosystems lead the table. Astro ($R = 0.89$) and MDX ($R = 0.86$) represent communities whose adoption is almost entirely post-2020; virtually every developer who has ever used these tools remains active by construction. Rust (0.84) and TypeScript (0.83) reflect the strong contemporary growth of systems-level and typed JavaScript development respectively.

At the lower end, Ruby (0.44) and Java (0.51) carry large populations of historical accounts from their respective peaks of GitHub adoption. MATLAB (0.50) sits near the overall median, consistent with a community of academic and engineering users whose engagement with public version control is more episodic than that of professional software developers.

The NONE category ($R = 0.36$) is substantially below the overall mean (0.589). This gap reflects two overlapping effects: inactive developers are less likely to maintain language-tagged repositories, and accounts with no public repositories at all—including spam, bot, and promotional accounts—are counted here. Further decomposition of this category is deferred to future work.

3.1 Compositional Asymmetry Between Active and Inactive Populations

The active and inactive populations differ not only in aggregate size but in their language composition. Among active accounts, Python (18.6%), TypeScript (13.2%), and JavaScript (9.4%) are the three most represented languages. Among inactive accounts, NONE accounts for 41.3% of the population—more than twice its share among active accounts (16.2%). TypeScript, by contrast, constitutes 13.2% of active accounts but only 3.8% of inactive accounts, illustrating the asymmetric concentration of active development in newer ecosystems.

3.2 The Objective-C Signal

Objective-C does not appear in Table 1 at the 150K snapshot because its sample size has not yet crossed the reporting threshold in this iteration of the analysis. At the 30K preliminary snapshot it exhibited $R = 0.15$ (9 active out of 59 total), the lowest ratio in the dataset by a substantial margin. This figure is consistent with the wholesale migration of Apple platform development from Objective-C to Swift following Swift’s introduction at WWDC 2014 and its open-sourcing in 2015. Developers whose most recently active repository is still tagged Objective-C constitute a population skewed heavily toward inactivity: those who have not migrated to Swift are, by and large, those who have stopped contributing publicly altogether. The ratio functions as a fossil record of a completed platform transition rather than a measure of an ongoing ecosystem.

4 Account Age and Developer Survival

Using account creation timestamps and most recent push timestamps, we estimate the fraction of developers remaining active as a function of account age. Table 2

presents the full survival curve across eighteen annual bins.

Account Age (years)	Active Fraction
0	0.631
1	0.656
2	0.648
3	0.620
4	0.608
5	0.609
6	0.597
7	0.574
8	0.556
9	0.540
10	0.527
11	0.524
12	0.538
13	0.565
14	0.606
15	0.633
16	0.661
17	0.680
18	0.632

Table 2: Active fraction by account age bin ($N = 154,283$).

4.1 A U-Shaped Survival Curve

The survival curve in Table 2 is not monotonically decreasing, as a simple dropout model would predict. Instead it exhibits a pronounced U-shaped form: the active fraction declines from approximately 0.631 at age zero through a minimum of 0.524 at age eleven, then *rises steadily* from age twelve onward, reaching 0.680 at age seventeen before a partial decline at age eighteen.

This pattern is unexpected and warrants careful interpretation. We propose the following account.

GitHub was founded in April 2008. Accounts that are now aged twelve to seventeen years were therefore created between approximately 2009 and 2014—the platform’s earliest adoption cohort. This population consists disproportionately of developers who were already deeply embedded in open source culture at the time of GitHub’s founding: early adopters, maintainers of established projects, and professionals for whom public version control was a durable practice rather than a transient experiment. Such developers are precisely those most likely to have sustained activity across a fifteen-year period. In other words, the oldest accounts that are still present in a follow network are not a random sample of early GitHub users: they are the *survivors*—a highly selected subpopulation of long-term contributors who were never likely to go inactive.

The intermediate cohort (ages five to eleven, corresponding to account creation roughly 2015–2021) catches the broad wave of GitHub adoption that followed the platform’s mainstream growth. This population is more heterogeneous: it includes students, hobbyists, and developers who created accounts for course projects or brief professional forays, many of whom have since become inactive. The trough in the survival curve at ages ten to eleven thus reflects the high churn characteristic of this mass-adoption wave.

The youngest cohort (ages zero to two, accounts created 2024–2026) shows a moderately high active fraction (0.63–0.66), but this figure is partially an artifact of left-censoring: accounts created recently and already gone inactive are less likely to appear in a follow network, because inactive accounts accumulate fewer followers over time.

Taken together, the U-shaped curve is better understood as a *selection artifact* than as a true survival phenomenon. It reflects the varying composition of different account-age cohorts as filtered through the follow-network sampling frame, rather than a literal rise in individual developer activity with age.

4.2 Absence of a Simple Half-Life

The preliminary 30K analysis suggested an approximate developer activity half-life of ten years, defined as the account age at which the active fraction first crosses 0.5. The 150K data substantially revise this picture: the active fraction does not cross 0.5 at any age bin in the current dataset. The minimum observed value is 0.524 at age eleven. This reflects the follow-network selection bias noted above: the seed account’s follow list is not a representative cross-section of all GitHub accounts, but a set of accounts that attracted sufficient attention to be followed, which selects for persistent activity. A simple half-life estimate derived from this curve would not be interpretable as a property of the general developer population.

5 Language-Specific Developer Lifetimes

We compute median observed developer lifetimes by language, defined as the median interval between account creation and most recent repository push over all accounts in the language with both timestamps present. Table 3 presents results for the twenty languages with the longest median lifetimes among those with at least 20 observations.

The languages at the top of Table 3 form a coherent profile: functional and systems languages (Erlang, Common Lisp, Clojure, Elixir, OCaml, Haskell, Scheme, F#), editor-specific scripting languages (Emacs Lisp, Vim Script), and languages associated with infrastructure and build tooling (Starlark, Go Template, Nix, LLVM, Just). These communities attract developers with deep technical commitments who maintain long-term engagement with their tools regardless of broader industry trends.

Zig (10.50 years, $n = 81$) is a notable exception: a young language (publicly released 2016) whose users nonetheless exhibit long measured lifetimes. This reflects

Language	Median Lifetime (years)	n
Erlang	12.65	22
Starlark	12.54	25
Common Lisp	11.75	53
Clojure	11.75	169
Elixir	11.62	163
Emacs Lisp	11.52	184
Go Template	10.90	53
OCaml	10.82	46
Haskell	10.63	98
Vim Script	10.55	257
Zig	10.50	81
Scheme	10.48	20
LLVM	10.37	43
Jinja	9.67	66
Nix	9.65	258
Nunjucks	9.55	36
Perl	9.46	162
F#	9.40	28
Rust	9.18	2560
Just	9.18	24

Table 3: Top 20 languages by median developer lifetime ($n \geq 20$).

the fact that Zig attracted experienced systems programmers who already held old GitHub accounts, rather than new entrants to the platform.

The contrast between median lifetime and activity ratio remains instructive at the 150K snapshot. Rust has both a long median lifetime (9.18 years, $n = 2,560$) and a high activity ratio (0.84), making it unusual: a large community that is both historically grounded and highly retained. Clojure and Elixir exhibit long lifetimes (11.75 and 11.62 years respectively) but more moderate activity ratios, consistent with stable, slowly growing communities of committed practitioners.

6 Discussion

The 150K snapshot substantially extends and in some cases revises the picture drawn from the 30K preliminary data.

The language activity ratios have stabilized: the rank ordering of major languages is consistent with the earlier snapshot, and the sample sizes for the largest languages (Python at 23,863; TypeScript at 14,401; JavaScript at 13,888) are sufficient to treat these ratios as reliable estimates of population-level activity rates. Small languages with fewer than a few hundred observations remain subject to significant sampling variability and should be interpreted with caution.

The survival curve is the most substantively novel finding at this scale. The U-shaped form—and in particular the recovery of active fractions among accounts

aged twelve to seventeen years—cannot be explained by a simple model of developer dropout. It requires acknowledging the compositional heterogeneity of the population: the oldest surviving accounts in a follow network are not typical early users, but a selected cohort of long-term contributors. This selection effect is intrinsic to the sampling frame and will persist in the full dataset. Correcting for it would require an estimate of the follower-count distribution conditional on account age and activity status, which the full dataset will make possible.

7 Future Work

The present analysis covers approximately 15% of the full follow list (50,040 batches total, 1,000,800 accounts). Completion of the crawl will enable:

- precise activity ratio estimates for smaller language communities currently below the reporting threshold,
- formal survival modeling (Weibull, log-normal, or mixture models) to characterize attrition across cohorts,
- disaggregated survival curves by language and account vintage to test whether the U-shaped pattern is uniform or language-specific,
- and analysis of the relationship between follower count and long-term activity, which would quantify the follow-network selection bias directly.

Because the dataset is developer-centered rather than repository-centered, it provides a perspective on open source community dynamics not available from existing large-scale GitHub datasets.

8 Crawl Architecture and Analysis Pipeline

8.1 Data Collection

The crawler issues paginated requests to the GitHub GraphQL API, traversing the follower list of a seed account whose follow graph exceeds one million accounts. Requests are batched at exactly 20 accounts per query. The total follow list spans approximately 1,000,800 accounts distributed across 50,040 batches; at the time of this report, approximately 7,700 batches ($\sim 15\%$) had been completed. Results are written incrementally to three output files: `following-activity.csv` (the primary structured record), `active-following.txt`, and `inactive-following.txt`, permitting intermediate analyses to be run against any partial snapshot without interrupting collection.

The CSV schema exposes the following fields per account:

Field	Description
<code>status</code>	ACTIVE or INACTIVE
<code>primary_language</code>	Language tag of most recently pushed repository
<code>created_at</code>	Account creation timestamp (ISO 8601)
<code>last_pushed_at</code>	Timestamp of most recent public repository push

An account is labeled **ACTIVE** if `last_pushed_at` falls within the twelve months preceding the crawl date; otherwise it is labeled **INACTIVE**. The crawler implements retry logic with attempt tracking, allowing transient API errors to be handled without restarting the full traversal.

8.2 Analysis Scripts

Three Python analysis scripts operate on the CSV output, invoked together by a shell driver (`run_analysis.sh`) that tees each script’s output to a corresponding file in an `analysis-results/` directory.

`analyse_activity.py` Computes the overall active/inactive split, per-language activity ratios (filtered to languages with at least 100 total accounts, sorted by total presence), and a year-resolution survival table derived from `created_at` timestamps. The approximate half-life is read from the survival table as the first bin whose active fraction falls below 0.5; as discussed in Section 4, this threshold is not crossed in the current dataset.

`analyse_languages.py` Separately tabulates the top languages by count among active and inactive accounts, reporting each language’s share of its respective subpopulation. This decomposition exposes compositional asymmetries not visible in the combined activity ratio.

`language_survival.py` Computes per-language median developer lifetimes as the median of $\{(\text{last_pushed_at} - \text{created_at})/365.25\}$ over all accounts with both timestamps present, excluding languages with fewer than 20 observations.

8.3 Rate Limiting and Projected Completion

At 20 accounts per GraphQL batch, the full follow list of $\sim 1,000,800$ accounts requires 50,040 batches. GitHub imposes a rate limit of 5,000 GraphQL points per hour on authenticated requests; the effective crawl rate is governed by the point cost of each batch query. The incremental write design ensures that each intermediate snapshot is self-consistent: the active/inactive classification threshold is applied at analysis time against the stored `last_pushed_at` field, so the definition of activity remains stable across repeated analyses of different snapshots.

9 Conclusion

Analysis of 154,283 accounts from a large GitHub follow network reveals systematic variation in developer activity across programming language ecosystems and across account-age cohorts. Astro, MDX, Rust, and TypeScript lead with activity ratios of 0.83–0.89; Ruby, Java, and the no-language (NONE) category trail at 0.36–0.51. The developer survival curve is U-shaped rather than monotonically decreasing, with the oldest accounts (aged 12–17 years) showing higher active fractions than middle-aged accounts, a pattern attributable to survivor selection within the follow-network sampling frame. Median developer lifetimes are longest in functional and infrastructure language communities and shorter in mass-adoption ecosystems.

Completion of the full one-million-account dataset will enable formal survival modeling, disaggregated cohort analysis, and direct quantification of the follow-network selection bias.